

T08. Subconsultas.

8.1 Introducción.....	1
8.2. Tipos de subconsultas.....	3
8.2.1. Subconsultas en la cláusula WHERE.....	3
8.2.2. Subconsultas en la cláusula HAVING.....	4
8.2.3. Subconsultas en la cláusula FROM.....	5
8.2.4. Subconsultas en la cláusula SELECT.....	6
8.3. Operadores usados en subconsultas.....	6
8.3.1. Operadores básicos de comparación.....	6
8.3.2. Subconsultas con ALL y ANY.....	7
8.3.3. Subconsultas con IN y NOT IN.....	8
8.3.4. Subconsultas con EXISTS y NOT EXISTS.....	9
8.4. Errores comunes.....	10
8.4.1. Número incorrecto de columnas en la subconsulta.....	10
8.4.2. Número incorrecto de filas en la subconsulta.....	10
8.5. Ejercicios.....	11
8.5.1. Subconsultas en general.....	11
8.5.2. Subconsultas con ALL y ANY.....	13
8.5.3. Subconsultas con IN y NOT IN.....	14
8.5.4. Subconsultas con EXISTS y NOT EXISTS.....	14

8.1 Introducción.

Una subconsulta es una consulta anidada dentro de otra consulta.

Debe tener en cuenta que no existe una única solución para resolver una consulta en SQL. En esta unidad vamos a estudiar cómo podemos resolverla utilizando subconsultas.

Para realizar los ejemplos utilizaremos las siguientes bases de datos, cada una con su correspondiente estructura:

- tiendadb.

```
mysql> DESC producto;
```

Field	Type	Null	Key	Default	Extra
codigo	int(10) unsigned	NO	PRI	NULL	auto_increment
nombre	varchar(100)	NO		NULL	
precio	double	NO		NULL	
codigo_fabricante	int(10) unsigned	NO	MUL	NULL	

4 rows in set (0.01 sec)

```
mysql> DESC fabricante;
```

Field	Type	Null	Key	Default	Extra
codigo	int(10) unsigned	NO	PRI	NULL	auto_increment
nombre	varchar(100)	NO		NULL	

2 rows in set (0.00 sec)

Database changed

```
mysql> select * from producto;
```

codigo	nombre	precio	codigo_fabricante
1	Disco duro SATA3 1TB	91.99	5
2	Memoria RAM DDR4 8GB	125	6
3	Disco SSD 1 TB	155.99	4
4	GeForce GTX 1050Ti	190	7
5	GeForce GTX 1080 Xtreme	760	6
6	Monitor 24 LED Full HD	207	1
7	Monitor 27 LED Full HD	250.99	1
8	Portátil Yoga 520	564	2
9	Portátil Ideapad 320	449	2
10	Impresora HP Deskjet 3720	64.99	3
11	Impresora HP Laserjet Pro M26nw	185	3
12	Portátil i7	680	4
13	Disco SSD 480 GB	87	8
14	Disco SSD 1TB	175	16
15	Disco SSD 256 GB	50	16

15 rows in set (0.00 sec)

```
mysql> select * from fabricante;
```

codigo	nombre
1	Asus
2	Lenovo
3	Hewlett-Packard
4	Samsung
5	Seagate
6	Crucial
7	Gigabyte
8	Apple
9	Xiaomi
11	IBM
12	Acer
16	Dell
30	Huawei

13 rows in set (0.00 sec)

- empeadodb:

```
mysql> desc empleado;
```

Field	Type	Null	Key	Default	Extra
codigo	int(11)	NO	PRI	NULL	auto_increment
nif	varchar(9)	NO		NULL	
nombre	varchar(30)	NO		NULL	
apellido1	varchar(30)	NO		NULL	
apellido2	varchar(30)	YES		NULL	
codigo_departamento	int(11)	YES	MUL	NULL	
codigo_jefe	int(11)	YES		NULL	

7 rows in set (0.01 sec)

```
mysql> desc departamento;
```

Field	Type	Null	Key	Default	Extra
codigo	int(11)	NO	PRI	NULL	auto_increment
nombre	varchar(30)	NO		NULL	
presupuesto	int(11)	YES		0	
gastos	int(11)	YES		1500	

4 rows in set (0.00 sec)

```
mysql> select * from empleado;
```

codigo	nif	nombre	apellido1	apellido2	codigo_departamento	codigo_jefe
1	20111222x	Julio	Romero	de Torres	1	3
2	20111333y	Antonio	Reyna	Manescau	6	4
3	20111444z	Juana	de Arcos	NULL	1	NULL
4	20111111a	Ada	Bayron	NULL	6	NULL
5	20222111a	Jimena	Burgos	Madrid	2	NULL
6	20222333b	Remedios	Varo	Sánchez	5	5
7	20222666c	Frida	Karlo	NULL	3	8
8	20222666c	Antonio	Candela	Heredia	3	NULL
9	20222777r	Maria	Triana	Fuentes	NULL	NULL
10	20222888h	Fernano	Fernan	Gómez	NULL	NULL
11	33000111s	Pepe	Ruiz	Santana	2	5
12	33000222d	David	Ruiz	Santana	3	8
13	33000333e	David	Ruiz	Ruiz	1	3
14	33000444r	Pepe	Ruiz	Ruiz	1	3
15	33888444w	Luis	Vari	NULL	6	4

15 rows in set (0.00 sec)

```
mysql> select * from departamento;
```

codigo	nombre	presupuesto	gastos
1	Sistemas	2000	3500
2	Desarrollo	2000	1500
3	Recursos Humanos	4000	3500
4	Ventas	4000	3500
5	Contabilidad	2000	1500
6	I+D	4000	7732

6 rows in set (0.00 sec)

- **ventasdb.**

```
mysql> desc cliente;
```

Field	Type	Null	Key
id	int(10) unsigned	NO	PRI
nombre	varchar(100)	NO	
apellido1	varchar(100)	NO	
apellido2	varchar(100)	YES	
ciudad	varchar(100)	YES	
categoría	int(10) unsigned	YES	

6 rows in set (0.01 sec)

```
mysql> desc comercial;
```

Field	Type	Null	Key
id	int(10) unsigned	NO	PRI
nombre	varchar(100)	NO	
apellido1	varchar(100)	NO	
apellido2	varchar(100)	YES	
comision	float	YES	

5 rows in set (0.00 sec)

```
mysql> desc pedido;
```

Field	Type	Null	Key
id	int(10) unsigned	NO	PRI
total	double	NO	
fecha	date	YES	
id_cliente	int(10) unsigned	NO	MUL
id_comercial	int(10) unsigned	NO	MUL

5 rows in set (0.00 sec)

```
mysql> select * from cliente;
```

id	nombre	apellido1	apellido2	ciudad
1	Aarón	Rivero	Gómez	Almería
2	Adela	Salas	Díaz	Granada
3	Adolfo	Rubio	Flores	Sevilla
4	Adrián	Suárez	NULL	Jaén
5	Marcos	Loyola	Méndez	Almería
6	Marta	Santana	Moreno	Cádiz
7	Pilar	Ruiz	NULL	Sevilla
8	Pepe	Ruiz	Santana	Huelva
9	Guillermo	López	Gómez	Granada
10	Daniel	Santana	Loyola	Sevilla

10 rows in set (0.00 sec)

```
mysql> select * from pedido;
```

id	total	fecha	id_cliente	id_comercial
1	150.5	2017-10-05	5	2
2	270.65	2016-09-10	1	5
3	65.26	2017-10-05	2	1
4	110.5	2016-08-17	8	3
5	948.5	2017-09-10	5	2
6	2400.6	2016-07-27	7	1
7	5760	2015-09-10	2	1
8	1983.43	2017-10-10	4	6
9	2480.4	2016-10-10	8	3
10	250.45	2015-06-27	8	2
11	75.29	2016-08-17	2	7
12	3045.6	2017-04-25	3	1
13	545.75	2019-01-25	6	1
14	145.82	2017-02-02	6	1
15	370.85	2019-03-11	1	5
16	2389.23	2019-03-11	1	5
17	300	2016-08-17	2	5

```
mysql> select * from comercial;
```

id	nombre	apellido1	apellido2
1	Daniel	Sáez	Vega
2	Juan	Gómez	López
3	Diego	Flores	Salas
4	Marta	Herrera	Gil
5	Antonio	Carpetero	Ortega
6	Manuel	Dominguez	Hernández
7	Antonio	Vega	Hernández
8	Alfredo	Ruiz	Flores

8 rows in set (0.00 sec)

8.2. Tipos de subconsultas.

El estándar SQL define tres tipos de subconsultas:

- **Subconsultas de tabla.** Son aquellas que devuelve una o varias columnas con una o varias filas.
- **Subconsultas de lista.** Son aquellas que devuelven una columna con una o varias filas.
- **Subconsultas de escalar.** Son aquellas que devuelven una columna y una fila. En definitiva, un valor que puede ser numérico, texto, fecha,...

Según en la cláusula en la que aparezcan:

- Subconsultas en WHERE.
- Subconsultas en HAVING.
- Subconsultas en FROM.
- Subconsultas en SELECT.

8.2.1. Subconsultas en la cláusula WHERE.

Se trata de consultas que requieren hacer otra consulta en la condición de WHERE. Por ejemplo, suponga que queremos conocer el nombre del producto que tiene el mayor precio. En este caso podríamos realizar una primera consulta para buscar cuál es el valor del precio máximo. Con ese resultado, (que sería de una subconsulta escalar, pues nos devuelve un valor) se consultaría el nombre del producto cuyo precio coincide con ese valor. La consulta sería la siguiente:

```
mysql> SELECT MAX(precio) FROM producto;
```

```
mysql> SELECT nombre FROM producto
```

```
WHERE precio=(SELECT MAX(precio) FROM producto);
```

```
mysql> SELECT nombre FROM producto WHERE precio =
      (SELECT MAX(precio) FROM producto);
+-----+
| nombre |
+-----+
| GeForce GTX 1080 Xtreme |
+-----+
1 row in set (0.00 sec)
```

En este caso sólo hay un nivel de anidamiento entre consultas pero pueden existir varios niveles de anidamiento.

Más ejemplos:

- Supongamos de la base de datos de empleadodb, queremos obtener un listado de los departamentos que tienen un gasto superior o igual al de “ventas:”

```
mysql> SELECT nombre, gastos FROM departamento
      WHERE gastos >= (SELECT gastos FROM departamento
      WHERE nombre= “ventas”);
```

- Listados de departamentos que tengas un gasto entre ventas e “I+D”:

```
mysql> SELECT nombre, gastos FROM departamento
      WHERE gastos >= (SELECT gastos FROM departamento
      WHERE nombre= “ventas”)
      AND
      gastos <= (SELECT gastos FROM departamento
      WHERE nombre= “I+D”);
```

8.2.2. Subconsultas en la cláusula HAVING

HAVING es una cláusula asociada a las consultas resumen o de agrupamiento. Luego la condición de HAVING comparará una función resumen con el resultado de la subconsulta.

Ejemplo: Devuelve un listado con todos los nombres de los fabricantes que proporcionan el mismo número de productos que el fabricante “Asus”.

La subconsulta ofrecerá un escalar: Número de productos de Asus. Al estar el nombre del fabricante en otra tabla es necesario hacer un join de producto con fabricante.

```
mysql> SELECT COUNT(*) FROM producto AS p join fabricante AS f
      ON p.codigo_fabricante = f.codigo WHERE f.nombre = “Asus”;

mysql> SELECT f.nombre, COUNT(p.codigo) AS cantidad
      FROM producto AS p JOIN fabricante AS f ON p.codigo_fabricante = f.codigo
      GROUP BY f.codigo HAVING cantidad =
      (SELECT COUNT(*) FROM producto AS p join fabricante AS f
      ON p.codigo_fabricante = f.codigo WHERE f.nombre = “Asus”);
```

En este caso COUNT(*) debe contar cuantos productos hay del fabricante Asus. Como se hace un JOIN de producto y fabricante se puede poner en lugar de “*” cualquier atributo de producto o fabricante que no sea NULL. Preferible “código” pues nos garantiza que no es NULL.

```
mysql> mysql> SELECT COUNT(*) FROM producto AS p JOIN fabricante AS f ON p.codigo_fabricante=f.codigo WHERE f.nombre="Asus";
+-----+
| COUNT(*) |
+-----+
|         2 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT COUNT(f.codigo) FROM producto AS p JOIN fabricante AS f ON p.codigo_fabricante=f.codigo WHERE f.nombre="Asus";
+-----+
| COUNT(f.codigo) |
+-----+
|         2 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT COUNT(p.codigo) FROM producto AS p JOIN fabricante AS f ON p.codigo_fabricante=f.codigo WHERE f.nombre="Asus";
+-----+
| COUNT(p.codigo) |
+-----+
|         2 |
+-----+
1 row in set (0.00 sec)
```

Esa consulta ofrece el número de productos de Asus. Para mostrar todos los fabricantes que ofrecen esa cantidad, la consulta anterior sería la subconsulta.

```
mysql> SELECT f.nombre, COUNT(p.codigo) as cantidad FROM producto as p JOIN fabricante AS f
ON p.codigo_fabricante=f.codigo
GROUP BY f.codigo HAVING cantidad=
(SELECT COUNT(p.codigo) FROM producto AS p JOIN fabricante AS f
ON p.codigo_fabricante=f.codigo WHERE f.nombre="Asus");
+-----+-----+
| nombre | cantidad |
+-----+-----+
| Asus   |         2 |
| Lenovo |         2 |
| Hewlett-Packard |         2 |
| Samsung |         2 |
| Crucial |         2 |
| Dell   |         2 |
+-----+-----+
6 rows in set (0.00 sec)
```

8.2.3. Subconsultas en la cláusula FROM

Detrás de FROM se especifica/n la/s tablas en las que se basará la consulta. Por lo tanto, en este caso el resultado de la subconsulta será una tabla desde la que se realizará la consulta principal.

Ejemplo: Devuelve una listado de todos los productos que tienen un precio mayor o igual al precio medio de cada fabricante. Empezaremos por obtener una consulta que nos devuelva una tabla con: código de fabricante y su precio medio, posteriormente haremos un JOIN con productos y mostraremos los productos cuyo precio sea mayor que el de la media de su fabricante.

```
mysql> SELECT codigo_fabricante AS fab, AVG(precio) AS media FROM producto
GROUP BY codigo_fabricante;
+-----+-----+
| fab | media |
+-----+-----+
| 1   | 228.995 |
| 2   | 506.5 |
| 3   | 124.995 |
| 4   | 417.995 |
| 5   | 91.99 |
| 6   | 442.5 |
| 7   | 190 |
| 8   | 87 |
| 16  | 112.5 |
+-----+-----+
9 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM producto AS p JOIN
(SELECT codigo_fabricante AS fab, AVG(precio) AS media FROM producto
GROUP BY codigo_fabricante) AS t
ON p.codigo_fabricante=t.fab WHERE p.precio>=t.media;
+-----+-----+-----+-----+-----+-----+
| codigo | nombre | precio | codigo_fabricante | fab | media |
+-----+-----+-----+-----+-----+-----+
| 7 | Monitor 27 LED Full HD | 250.99 | 1 | 1 | 228.995 |
| 8 | Portátil Yoga 520 | 564 | 2 | 2 | 506.5 |
| 11 | Impresora HP Laserjet Pro M26nw | 185 | 3 | 3 | 124.995 |
| 12 | Portátil i7 | 680 | 4 | 4 | 417.995 |
| 1 | Disco duro SATA3 1TB | 91.99 | 5 | 5 | 91.99 |
| 5 | GeForce GTX 1080 Xtreme | 760 | 6 | 6 | 442.5 |
| 4 | GeForce GTX 1050Ti | 190 | 7 | 7 | 190 |
| 13 | Disco SSD 480 GB | 87 | 8 | 8 | 87 |
| 14 | Disco SSD 1TB | 175 | 16 | 16 | 112.5 |
+-----+-----+-----+-----+-----+-----+
9 rows in set (0.01 sec)
```

8.2.4. Subconsultas en la cláusula SELECT

La subconsulta ha de devolver un escalar que se mostrará o con el que se realizará una expresión en los elementos del SELECT de la consulta principal.

Ejemplo: Realiza una consulta en la que se muestren los datos de todos los productos comparados con la media de cada fabricante. La media de cada fabricante es un valor que se ha de calcular para cada producto a mostrar. Obviamente, todos los productos del mismo fabricante tendrán la misma media.

```
mysql> SELECT codigo, nombre, precio,
(select avg(t.precio) from producto as t
where p.codigo_fabricante=t.codigo_fabricante) as "Media fabricante",
codigo_fabricante as fab
from producto as p order by p.codigo_fabricante;
```

codigo	nombre	precio	Media fabricante	fab
6	Monitor 24 LED Full HD	207	228.995	1
7	Monitor 27 LED Full HD	250.99	228.995	1
8	Portátil Yoga 520	564	506.5	2
9	Portátil Ideapad 320	449	506.5	2
10	Impresora HP Deskjet 3720	64.99	124.995	3
11	Impresora HP Laserjet Pro M26nw	185	124.995	3
3	Disco SSD 1 TB	155.99	417.995	4
12	Portátil i7	680	417.995	4
1	Disco duro SATA3 1TB	91.99	91.99	5
2	Memoria RAM DDR4 8GB	125	442.5	6
5	GeForce GTX 1080 Xtreme	760	442.5	6
4	GeForce GTX 1050Ti	190	190	7
13	Disco SSD 480 GB	87	87	8
14	Disco SSD 1TB	175	112.5	16
15	Disco SSD 256 GB	50	112.5	16

15 rows in set (0.01 sec)

8.3. Operadores usados en subconsultas.

Los operadores que podemos usar en las subconsultas son los siguientes:

- Operadores básicos de comparación (>, >=, <, <=, !=, <>, =).
- Predicados ALL y ANY.
- Predicado IN y NOT IN.
- Predicado EXISTS y NOT EXISTS.

8.3.1. Operadores básicos de comparación

Los operadores básicos de comparación (>, >=, <, <=, !=, <>, =) se pueden usar cuando queremos comparar una expresión con el valor que devuelve una subconsulta.

Los operadores básicos de comparación los vamos a utilizar para realizar comparaciones con **subconsultas que devuelven un único valor, es decir, de tipo escalar (una columna y una fila).**

Ejemplo: Devuelve todos los productos de la base de datos que tienen un precio mayor o igual al producto más caro del fabricante Asus.

```
mysql> SELECT * FROM producto WHERE precio >=
      (SELECT MAX(precio) FROM producto WHERE codigo_fabricante =
      (SELECT codigo FROM fabricante WHERE nombre = "Asus"));
```

codigo	nombre	precio	codigo_fabricante
5	GeForce GTX 1080 Xtreme	760	6
7	Monitor 27 LED Full HD	250.99	1
8	Portátil Yoga 520	564	2
9	Portátil Ideapad 320	449	2
12	Portátil i7	680	4

5 rows in set (0.00 sec)

La consulta anterior también se puede hacer utilizando un JOIN:

```
mysql> SELECT * FROM producto WHERE precio >=
      (SELECT MAX(p.precio) FROM producto AS p JOIN fabricante AS f
      ON p.codigo_fabricante = f.codigo WHERE f.nombre = "Asus"),
```

8.3.2. Subconsultas con ALL y ANY

ALL y ANY se utilizan con los operadores de comparación (>, >=, <, <=, !=, <>, =) y nos permiten comparar una expresión con el conjunto de valores que devuelve una subconsulta.

ALL y ANY los vamos a utilizar para realizar comparaciones con **subconsultas que pueden devolver varios valores, es decir, una columna y varias filas**.

Ejemplo: Podemos escribir la consulta que devuelve todos los productos de la base de datos que tienen un precio mayor o igual al producto más caro del fabricante Asus, haciendo uso de ALL. En primer lugar habría que preparar una consulta que devuelva los precios de los productos de Asus, esta sería la subconsulta. La consulta principal será la de los productos que tengan un precio mayor o igual a **todos** (ALL) los de la lista de la subconsulta.

```
mysql> SELECT * FROM producto WHERE precio >= ALL
      (SELECT p.precio FROM producto AS p
      JOIN fabricante AS f ON p.codigo_fabricante=f.codigo
      WHERE f.nombre = "Asus");
```

codigo	nombre	precio	codigo_fabricante
5	GeForce GTX 1080 Xtreme	760	6
7	Monitor 27 LED Full HD	250.99	1
8	Portátil Yoga 520	564	2
9	Portátil Ideapad 320	449	2
12	Portátil i7	680	4

5 rows in set (0.01 sec)

El resultado sería el mismo si se utiliza la función MAX:

```
mysql> SELECT * FROM producto WHERE precio >=
      (SELECT MAX(p.precio) FROM producto AS p
      JOIN fabricante AS f ON p.codigo_fabricante=f.codigo
      WHERE f.nombre = "Asus");
```

La palabra reservada SOME es un alias de ANY. Por lo tanto, las siguientes consultas devolverían el mismo resultado:

```
mysql> SELECT s1 FROM t1 WHERE s1 <> ANY (SELECT s1 FROM t2)
```

```
mysql> SELECT s1 FROM t1 WHERE s1 <> SOME (SELECT s1 FROM t2)
```

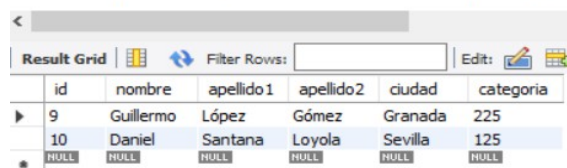
8.3.3. Subconsultas con IN y NOT IN

IN y NOT IN nos permiten comprobar si un valor está o no incluido en un conjunto de valores, que puede ser el conjunto de valores que devuelve una subconsulta.

IN y NOT IN los vamos a utilizar para realizar comparaciones con **subconsultas que pueden devolver varios valores, es decir, una columna y varias filas**.

Ejemplo: Devuelve un listado de los clientes que no han realizado ningún pedido, base de datos ventasdb. La subconsulta ofrece una lista de clientes que sí han realizado pedidos.

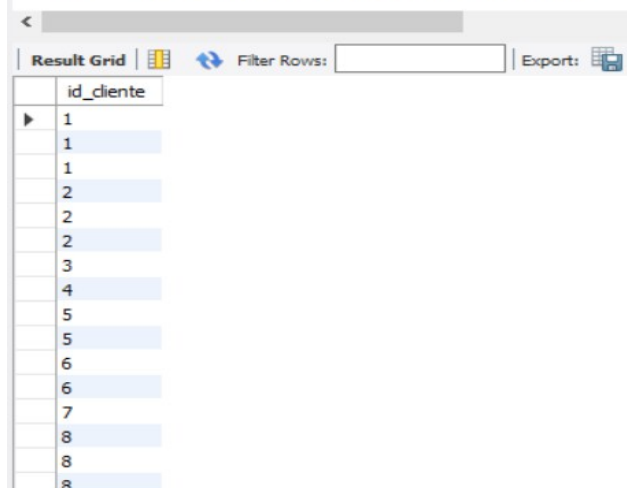
```
1 • SELECT * FROM cliente WHERE id NOT IN
2   (SELECT id_cliente FROM ventasdb.pedido);
```



id	nombre	apellido1	apellido2	ciudad	categoria
9	Guillermo	López	Gómez	Granada	225
10	Daniel	Santana	Loyola	Sevilla	125
NULL	NULL	NULL	NULL	NULL	NULL

Obsérvese que el resultado de la subconsulta es:

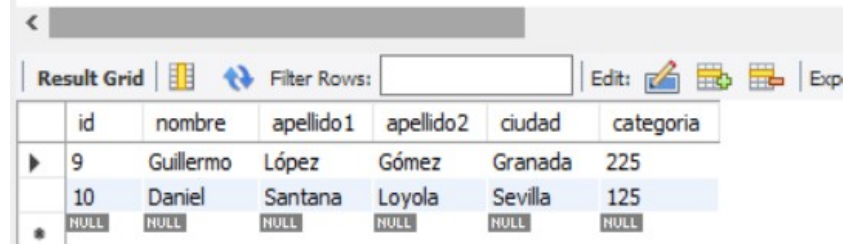
```
1 • SELECT id_cliente FROM ventasdb.pedido;
```



id_cliente
1
1
1
2
2
2
3
4
5
5
6
6
7
8
8
8

Con todos esos valores se ha de comparar el “id” de cada cliente “WHERE id NOT IN..”:17 comparaciones. Obtendríamos un código mucho más eficiente si se evitan las repeticiones con DISTINCT, en ese caso se harían 8 comparaciones y el resultado es el mismo:

```
1 • SELECT * FROM cliente WHERE id NOT IN
2   (SELECT DISTINCT id_cliente FROM ventasdb.pedido);
```



id	nombre	apellido1	apellido2	ciudad	categoria
9	Guillermo	López	Gómez	Granada	225
10	Daniel	Santana	Loyola	Sevilla	125
NULL	NULL	NULL	NULL	NULL	NULL

Cuando estamos trabajando con subconsultas, “IN” y “= ANY” realizan la misma función. Por lo tanto, las siguientes consultas devolverían el mismo resultado:

```
mysql> SELECT s1 FROM t1 WHERE s1 = ANY (SELECT s1 FROM t2);  
mysql> SELECT s1 FROM t1 WHERE s1 IN (SELECT s1 FROM t2);
```

Ocurre lo mismo con “NOT IN” y “<> ALL”. Por lo tanto, las siguientes consultas devolverían el mismo resultado:

```
mysql> SELECT s1 FROM t1 WHERE s1 <> ALL (SELECT s1 FROM t2);  
mysql> SELECT s1 FROM t1 WHERE s1 NOT IN (SELECT s1 FROM t2);
```

Importante:

- Cuando hay **algún valor NULL** en el resultado de la consulta interna, la consulta externa utilizando **NOT IN** no devuelve ningún resultado.
- En el caso de **ALL**: Si la subconsulta no devuelve ningún valor, el operador ALL devuelve el valor verdadero. Supongamos una subconsulta de presupuestos de departamentos con superior o igual a 10000:

```
mysql> SELECT presupuesto FROM departamento WHERE presupuesto >= 10000;
```

El resultado es vacío, pues el máximo es 4000.

Si queremos obtener un listado de departamentos con máximo presupuestos superior a 10000 usando ALL:

```
mysql> SELECT codigo, nombre FROM departamento WHERE presupuesto >= ALL  
      (SELECT presupuesto FROM departamento WHERE presupuesto >= 10000);
```

Obsérvese que se muestra un listado de todos los departamentos, cuando no debería mostrar ninguno, pues no hay ninguno que iguale o supere los 10000 euros.

- Para tener un código más eficiente, debemos asegurarnos que en la subconsulta que devuelve el listado de elementos a comparar, no haya repeticiones. DISTINCT resuelve esa situación.

Ejemplo: Devuelve un listado con el nombre de los departamentos que no tienen empleados asociados. Utiliza NOT IN.

El número de comparaciones a realizar muestra claramente menor, eso implica eficiencia en la ejecución.

Por otro lado se evita NULL, que será fuente de error en las comparaciones con IN y NOT IN.

```

7 • SELECT codigo_departamento FROM empleado;
Result Grid | Filter Rows: | Export:
codigo_departamento
1 NULL
2 1
3 1
4 1
5 2
6 2
7 3
8 3
9 3
10 6
11 6
12 6

6 • SELECT nombre FROM departamento WHERE codigo NOT IN
7 (SELECT codigo_departamento FROM empleado);
Result Grid | Filter Rows: | Export: | Wrap Cell Content
nombre
1

```

```

7 • SELECT DISTINCT codigo_departamento FROM empleado
8 WHERE codigo_departamento IS NOT NULL;
Result Grid | Filter Rows: | Export: | Wrap Cell Content
codigo_departamento
1 1
2 2
3 3
4 6

```

```

7 • SELECT nombre FROM departamento WHERE codigo NOT IN
8 (SELECT DISTINCT codigo_departamento FROM empleado
9 WHERE codigo_departamento IS NOT NULL);
Result Grid | Filter Rows: | Export: | Wrap Cell Content
nombre
1 Ventas
2 Contabilidad

```

En el primer caso, al tener la subconsulta dos resultados NULL, cuando tiene que compara (NOT IN) con los códigos de los departamentos de la lista de la subconsulta no sabe como valorar la comparación con el NULL por eso devuelve 0 resultados.

En el segundo caso, se protege la consulta interna para que no devuelva NULL y se añade la cláusula DISTINCT que acelera el proceso pasando de 13 a 4 comparaciones por cada departamento.

Repite la consulta (listado de departamentos que no tienen empleados) pero utilizando: ALL

```

mysql> SELECT codigo, nombre FROM departamento
      WHERE codigo != ALL
      (SELECT DISTINCT codigo_departamento FROM empleado WHERE
      codigo_departamento IS NOT NULL);

```

Listado de departamentos que tienen empleados utilizando IN y ANY.

```

mysql> SELECT codigo, nombre FROM departamento
      WHERE codigo in (SELECT DISTINCT codigo_departamento FROM empleado);

mysql> SELECT codigo, nombre FROM departamento
      WHERE codigo = ANY (SELECT DISTINCT codigo_departamento FROM
      empleado);

```

8.3.4. Subconsultas con EXISTS y NOT EXISTS

Con esta cláusula EXISTS, basta con que lo que venga detrás tenga algún resultado para darlo por válido o no.

Ejemplo: Devuelve un listado de los clientes que no han realizado ningún pedido.

```
mysql> SELECT CONCAT_WS(", ", CONCAT_WS(" ", c.apellido1, c.apellido2), c.nombre)
        AS cliente FROM cliente AS c
        WHERE NOT EXISTS (SELECT * FROM pedido AS p WHERE p.id_cliente=c.id);
```

cliente
López Gómez, Guillermo
Santana Loyola, Daniel

2 rows in set (0.00 sec)



Debemos tener en cuenta que dentro del paréntesis que va detrás del EXISTS se debe hacer referencia a algún contenido de fuera de este de otra forma no tendría efecto, pues si ofrece alguna fila siempre la ofrecerá al no intervenir ningún elemento de fuera; y si no la ofrece pues nunca la ofrecerá.

Otros ejemplos:

Listado de departamentos que no tienen empleados.

```
mysql> SELECT d.codigo, d.nombre FROM departamento d WHERE NOT EXISTS
        (SELECT * from empleado where codigo_departamento=d.codigo);
```

Listado de departamentos con empleados.

```
mysql> SELECT d.codigo, d.nombre FROM departamento d WHERE EXISTS
        (SELECT * from empleado where codigo_departamento=d.codigo);
```

8.4. Errores comunes.

8.4.1. Número incorrecto de columnas en la subconsulta.

Cuando una subconsulta ofrece más de una columna y donde se espera el resultado solo se espera una columna nos daría este tipo de error.

Ejemplo:

```
mysql> SELECT (SELECT column1, column2 FROM t2) FROM t1;
```

La subconsulta solo debería mostrar una única columna.

El error que aparecería:

```
ERROR 1241 (ER_OPERAND_COL) SQLSTATE = 21000 Message = "Operand should
contain 1 column(s)"
```

8.4.2. Número incorrecto de filas en la subconsulta.

Igualmente si el resultado de una subconsulta se espera que sea una única fila y nos ofrece más de una también daría error, pues no sabría cual de ellas utilizar. Supongamos que estamos haciendo una comparación con el valor que ofrezca una subconsulta, si esta ofrece más de un valor, ¿Con cual de ellos se compara?

Ejemplo:

```
mysql> SELECT * FROM t1 WHERE column1 = (SELECT column1 FROM t2);
```

La consulta anterior sólo se podrá ejecutar cuando la consulta interna SELECT column1 FROM t2 devuelva una única fila.

El error que aparecería:

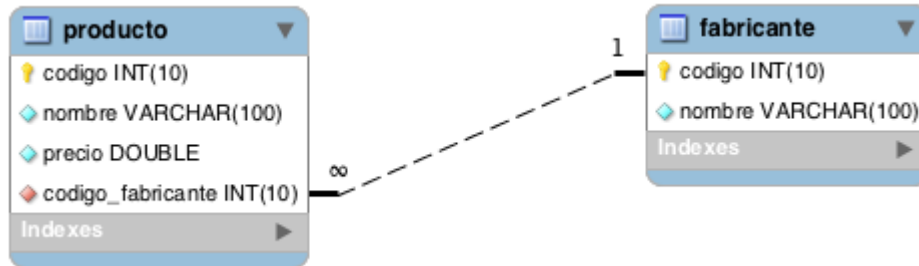
```
ERROR 1242 (ER_SUBSELECT_NO_1_ROW) SQLSTATE = 21000 Message =
"Subquery returns more than 1 row"
```

8.5. Ejercicios.

8.5.1. Subconsultas en general.

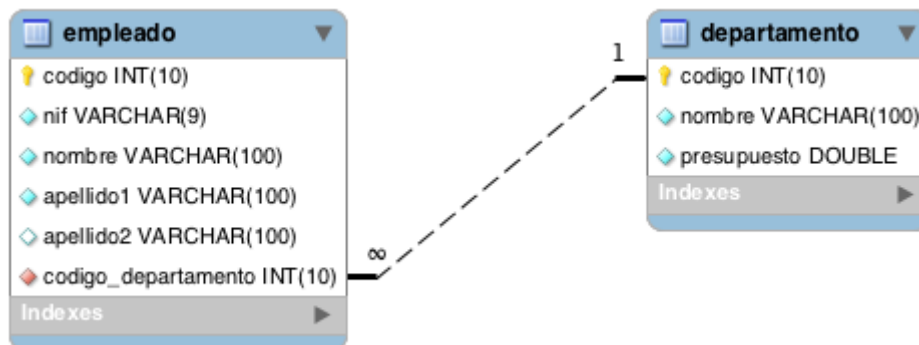
Realiza los ejercicios sin utilizar INNER JOIN.

Base de datos: tiendadb.



1. Devuelve todos los productos del fabricante “Lenovo”.
2. Devuelve todos los datos de los productos que tienen el mismo precio o superior al producto más caro del fabricante “Asus”.
3. Lista el nombre del producto más caro del fabricante Lenovo.
4. Lista el nombre del producto más barato del fabricante Hewlett-Packard.
5. Lista todos los productos del fabricante Asus, Hewlett-Packard, Lenovo que tienen un precio superior al precio medio de todos sus productos.

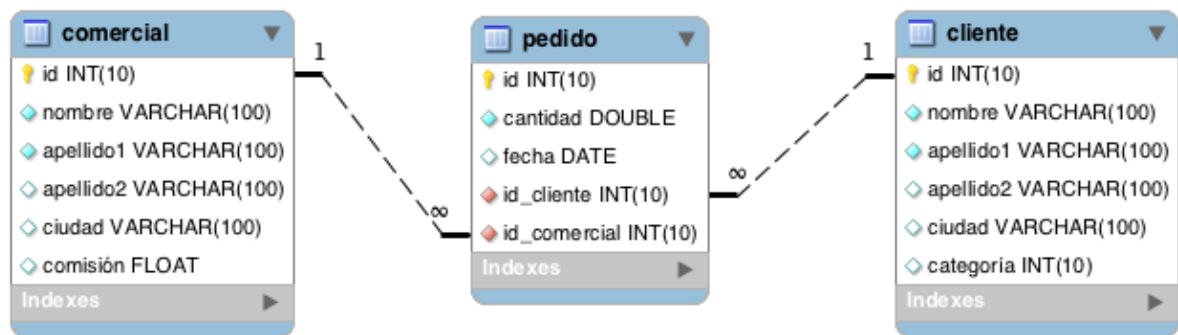
Base de datos: empleadodb.



1. Devuelve un listado con todos los empleados que tiene el departamento de Sistemas.
2. Devuelve el nombre del departamento con mayor presupuesto y la cantidad que tiene asignada.
3. Devuelve el nombre del departamento con menor presupuesto y la cantidad que tiene asignada.

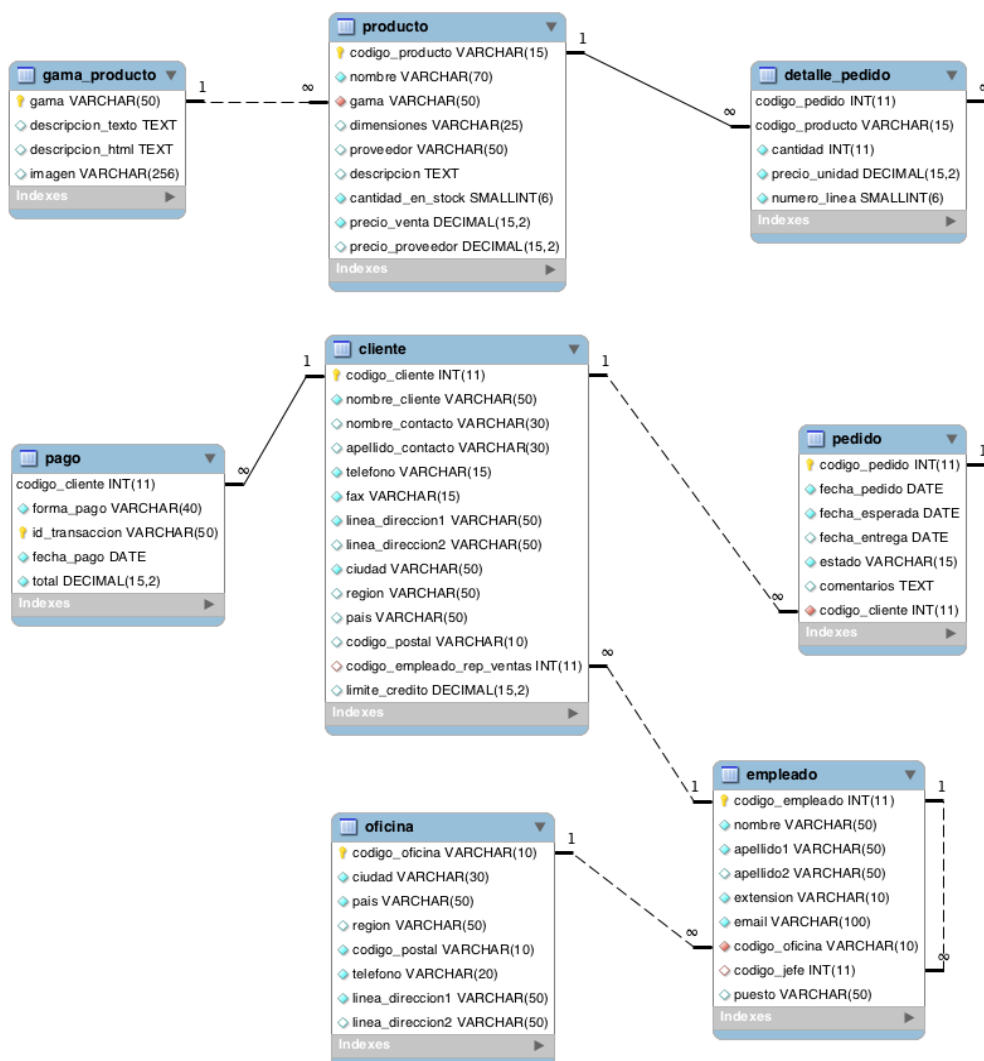
Base de datos: ventasdb.

1. Devuelve un listado con todos los pedidos que ha realizado Adela Salas Díaz.



2. Devuelve el número de pedidos en los que ha participado el comercial Daniel Sáez Vega.
3. Devuelve los datos del cliente que realizó el pedido más caro en el año 2019.
4. Devuelve la fecha y la cantidad del pedido de menor valor realizado por el cliente Pepe Ruiz Santana.
5. Devuelve un listado con los datos de los clientes y los pedidos, de todos los clientes que han realizado un pedido durante el año 2017 con un valor mayor o igual al valor medio de los pedidos realizados durante ese mismo año.

Base de datos: jardineria.



1. Devuelve el nombre del cliente con mayor límite de crédito.
2. Devuelve el nombre del producto que tenga el precio de venta más caro.
3. Devuelve el nombre del producto del que se han vendido más unidades. (Tenga en cuenta que tendrá que calcular cuál es el número total de unidades que se han vendido de cada producto a partir de los datos de la tabla detalle_pedido. Una vez que sepa cuál es el código del producto, puede obtener su nombre fácilmente).
4. Los clientes cuyo límite de crédito sea mayor que los pagos que haya realizado.
5. Devuelve el producto que más unidades tiene en stock.
6. Devuelve el producto que menos unidades tiene en stock.
7. Devuelve el nombre, los apellidos y el email de los empleados que están a cargo de Alberto Soria.

8.5.2. Subconsultas con ALL y ANY.

Base de datos: tiendadb.

1. Devuelve el producto más caro que existe en la tabla producto sin hacer uso de MAX, ORDER BY ni LIMIT.
2. Devuelve el producto más barato que existe en la tabla producto sin hacer uso de MIN, ORDER BY ni LIMIT.
3. Devuelve los nombres de los fabricantes que tienen productos asociados.
4. Devuelve los nombres de los fabricantes que no tienen productos asociados.

Base de datos: empleadodb.

1. Devuelve el nombre del departamento con mayor presupuesto y la cantidad que tiene asignada. Sin hacer uso de MAX, ORDER BY ni LIMIT.
2. Devuelve el nombre del departamento con menor presupuesto y la cantidad que tiene asignada. Sin hacer uso de MIN, ORDER BY ni LIMIT.
3. Devuelve los nombres de los departamentos que tienen empleados asociados.
4. Devuelve los nombres de los departamentos que no tienen empleados asociados.

Base de datos: ventasdb.

1. Devuelve el pedido más caro que existe en la tabla pedido si hacer uso de MAX, ORDER BY ni LIMIT.
2. Devuelve un listado de los clientes que no han realizado ningún pedido.
3. Devuelve un listado de los comerciales que no han realizado ningún pedido.

8.5.3. Subconsultas con IN y NOT IN.

Base de datos: tiendadb.

1. Devuelve los nombres de los fabricantes que tienen productos asociados.
2. Devuelve los nombres de los fabricantes que no tienen productos asociados.

Base de datos: empleadodb.

1. Devuelve los nombres de los departamentos que tienen empleados asociados.
2. Devuelve los nombres de los departamentos que no tienen empleados asociados.

Base de datos: ventadb.

1. Devuelve un listado de los clientes que no han realizado ningún pedido.
2. Devuelve un listado de los comerciales que no han realizado ningún pedido.

8.5.4. Subconsultas con EXISTS y NOT EXISTS.

Base de datos: tiendadb.

1. Devuelve los nombres de los fabricantes que tienen productos asociados.
2. Devuelve los nombres de los fabricantes que no tienen productos asociados.

Base de datos: empleadodb.

1. Devuelve los nombres de los departamentos que tienen empleados asociados.
2. Devuelve los nombres de los departamentos que tienen empleados asociados.

Base de datos: ventasdb.

1. Devuelve un listado de los clientes que no han realizado ningún pedido.
2. Devuelve un listado de los comerciales que no han realizado ningún pedido.

Base de datos: Jardinería.

1. Devuelve un listado que muestre solamente los clientes que no han realizado ningún pago.
2. Devuelve un listado que muestre solamente los clientes que sí han realizado algún pago.
3. Devuelve un listado de los productos que nunca han aparecido en un pedido.
4. Devuelve un listado de los productos que han aparecido en un pedido alguna vez.